Assignment 2: A Basic Sequential Web Proxy
Due: March 21, 11:30 PM

1. Introduction

A web proxy is a program that acts as a middleman between a web server and browser. Instead of contacting the server directly to get a web page, the browser contacts the proxy, which forwards the request on to the server. When the server replies to the proxy, the proxy sends the reply onto the browser. Proxies are used for many purposes. Sometimes proxies are used in firewalls, such that the proxy is the only way for a browser inside the firewall to contact a server outside. The proxy may do translation on the page, for instance, to make it viewable on a web-enabled cell phone.

In this assignment, you will write a basic sequential web proxy. You will set up the proxy to accept a request, forward the request to the server, and return the result back to the browser, keeping a log of such requests in a disk file. In this part, you will learn how to write programs that interact with each other over a network (socket programming), as well as some basic HTTP.

2. Handout Instruction

Start by downloading **proxy-handout.tar** from Canvas to a directory in which you plan to do your work.

The tar file is a file archive that can be extracted on UNIX machines with the command

tar xvf proxy-handout.tar

This will cause a number of files to be unpacked in the directory. The three files you will be modifying and turning in are proxy.c, csapp.c, and csapp.h. You may add any files you wish to this directory as you will be submitting the entire directory (Make sure you update the Makefile as you add any new files). The proxy.c file should eventually contain the bulk of the logic for your proxy. The csapp.c and csapp.h files are described in your textbook. The csapp.c file contains error handling wrappers and helper functions such as the RIO functions, the open clientfd function, and the open listenfd function. It is not mandatory to use the helper functions in csapp.c. You can write yours if you feel so. Include in your report if you make any changes in the csapp.h and/or csapp.c file.

3. Implementation

The basic sequential web proxy will handle requests one at a time. When started, your proxy should open a socket and listen for connection requests on the port number that is passed in on the command line. (See the section “Port Numbers” below.)

When the proxy receives a connection request from a client (typically a web browser), the proxy should accept the connection, read the request, verify that it is a valid HTTP

request, and parse it to determine the server that the request was meant for. It should then open a connection to that server, send it the request, receive the reply, and forward the reply to the browser.

Notice that, since your proxy is a middleman between client and server, it will have elements of both. It will act as a server to the web browser and as a client to the web server.

3.1. Processing HTTP Requests

When an end user enters a URL such as *http://www.yahoo.com/news.html* into the address bar of the browser, the browser sends an HTTP request to the proxy that begins with a line looking something like this:

GET http://www.yahoo.com/news.html HTTP/1.0

In this case the proxy will parse the request, open a connection to www.yahoo.com, and then send an HTTP request starting with a line of the form:

GET /news.html HTTP/1.0

to the server www.yahoo.com. Please note that all lines end with a carriage return '\r' followed by a line feed '\n', and that HTTP request headers are terminated with an empty line. Since a port number was not specified in the browser's request, in this example the proxy connects to the default HTTP port (port 80) on the server. The web browser may specify a port that the web server is listening on, if it is different from the default of 80. This is encoded in a URL as follows:

http://www.example.com:8080/index.html

The proxy, on seeing this URL in a request, should connect to the server www.example.comon port 8080. The proxy then simply forwards the response from the server on to the browser.

3.1.1. Connection Handling

By default, your proxy should close the connection after forwarding a request and receiving a response. This follows the standard behavior of HTTP/1.0, where each request uses a separate connection.

- If the client request includes Connection: keep-alive, the proxy should ignore it and close the connection.
- If the server response includes Connection: keep-alive, the proxy should still close the connection after forwarding the response.
- Your proxy does not need to implement connection reuse.

3.1.2. Looking Ahead: Persistent Connections

In this assignment, your proxy closes the connection after each request (following HTTP/1.0 behavior). However, modern web browsers and servers often use persistent connections (Connection: keep-alive) to reuse the same connection for multiple

requests, reducing the overhead of repeatedly opening and closing sockets.

- Your proxy does not need to support persistent connections for this assignment.
- If you're interested in exploring this topic, Assignment 3 will introduce connection persistence and allow you to implement it efficiently.

Reminder: Correct Port Handling

- If the request URL includes a port (e.g., http://www.example.com:8080/), extract and use it.
- If no port is specified, default to port 80 (standard HTTP).
- Failing to handle this properly will result in incorrect proxy behavior and lost points!

TIP: Read the brief section in the CSAPP textbook on “HTTP transactions” to better understand the format of the HTTP requests your proxy should send to a server.

NOTE: Be sure to parse out the port number from the URL. We will be testing this. If the port is not explicitly stated, use the default port of port 80.

❖ **Looking Ahead: HTTPS Support**

This assignment only supports plain HTTP requests.

HTTPS requests (e.g., https://www.google.com/) will not work because they require a different approach.

In Assignment 3, you will have the option to add HTTPS tunneling support using the CONNECT method.

3.1.3. Common Mistakes to Avoid

When implementing your proxy, be mindful of the following common errors:

1) Incorrectly Parsing the Port Number

- If a URL includes a port (e.g., http://www.example.com:8080/index.html), your proxy must extract and use it.
- If no port is specified, the proxy should default to port 80.

2) Incorrect HTTP Request Formatting

- Your proxy must rewrite the client’s request to ensure it matches the format required by the destination server.
- The first line should follow:

  ```
  GET /path/to/resource HTTP/1.1
  ```

- For example:

  rewrite

  GET https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages HTTP/1.1

  to

  GET /en-US/docs/Web/HTTP/Messages HTTP/1.1

- The HOST header must be included if it is not included in the original request.

3) Failure to Handle Large Responses

- Some web pages return large responses. Your proxy should read and forward the entire response, not just the first chunk.

4) Logging Only Requests, Not Responses

- Your log should include the response size, not just the request details.

3.2. Port Numbers

Every server on the Internet waits for client connections on a well-known port. The exact port number varies from Internet service to service. The clients of your proxy (your browser for example), will need to be told not just the hostname of the machine running the proxy, but also the port number on which it is listening for connections.

Your proxy should accept a command-line argument that gives the port number on which it should listen for connection requests. For example, the following command runs a proxy listening on port 15213:

unix> ./proxy 15213

You will need to specify a port each time you wish to test the code you've written.

3.3. Logging

You will need to log each request to a log file. Your proxy should keep track of all requests in a log file named proxy.log. Each log file entry should be in the form:

Date: browserIP URL size

where browserIP is the IP address of the browser, URL is the URL asked for, size is the size in bytes of the object that was returned. For instance:

Sun 27 Oct 2002 02:51:02 EST: 128.2.111.38 http://www.cs.cmu.edu/ 34314

Note that size is essentially the number of bytes received from the endserver, from the time the connection is opened to the time it is closed. Only requests that are met by a response from an end server should be logged. We have provided the function format_log_entry in proxy.c to create a log entry in the required format.

3.3.1. Log File Example

If a client from 192.168.1.5 requests http://www.example.com/index.html and receives a response of 10,512 bytes, the log entry should be:

Wed 26 Feb 2025 14:30:10 UTC: 192.168.1.5 http://www.example.com/index.html 10512

- Ensure the log format follows this structure exactly, as we will check this in grading.
- Only successful requests (those with valid server responses) should be logged.

4. Testing Your Proxy

For this assignment, there is no script to check your implementation. You will have to come up with your own tests to help you debug your code and decide when you have a correct implementation. Below is way by which you can debug your proxy, to help you get started.

- ❖ You can test the working by any internet browser. You will need to configure the HTTP proxy settings for that browser. For example, in Mozilla, you can set the appropriate proxy setting at Options->Advanced->Network->Connection->Configure how Firefox connects to the Internet. Click the Settings button and set only your HTTP proxy (using manual configuration). The server will be whatever machine your proxy is running on, and the port is the same as the one you passed to the proxy when you ran it.
- ❖ Use print statements to trace the flow of information between the client, proxy and the server.

5. Tips

We strongly recommend that you tackle this proxy problem in the following steps. This should help you break the problem down into more manageable pieces that you can test individually. Working versions of these will also count for partial credit if the complete proxy is not finished.

1. Write a proxy server that accepts a connection from a web browser and simply prints the entire request message from the web browser on the screen.
2. Extend the server from (1) to extract the browser IP and URL information from the request message. Open a log file and write this information into a file, instead of printing the message on the screen. Use the provided function to format the output. Set size to 0, since it is unknown at this point.
3. Write a separate client that takes a URL as input and prints out the web server response for that URL as well as the size of the response in bytes.
4. Combine the server and client into a single program. Instead of having the client print the output on the screen, feed it back to the web browser.

6. Submission and Demo Instructions

You will need to submit your complete project directory on Canvas.

- Create a gzipped tar ball that includes all your source code.
- The filename for the tarball must be **username-proxy.tar.gz** where username is your loginname on the EIT(netid) system.
- All your files in the tarball **must** be in a directory, named **username-proxy**.

- Also, submit a very brief report in a plain text file about the functions that you implemented. A line about what the function does is enough.
- The submission will be due on the date mentioned on Canvas.